

FASE 1 - Fundamentos de Inteligência Artificial - aula 4 - IA Generativa

URL da página da aula na plataforma

<https://on.fiap.com.br/mod/conteudoshtml/view.php?id=520838&c=14006&sesskey=03CWUSEStJ>

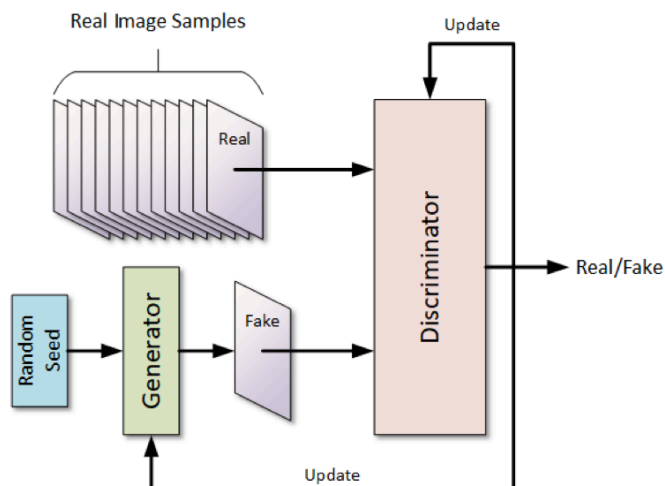
URL do PDF da aula

<https://on.fiap.com.br/mod/conteudospdf/view.php?c=14006&id=520842>

Iniciando em 02-09-2025

Anotações

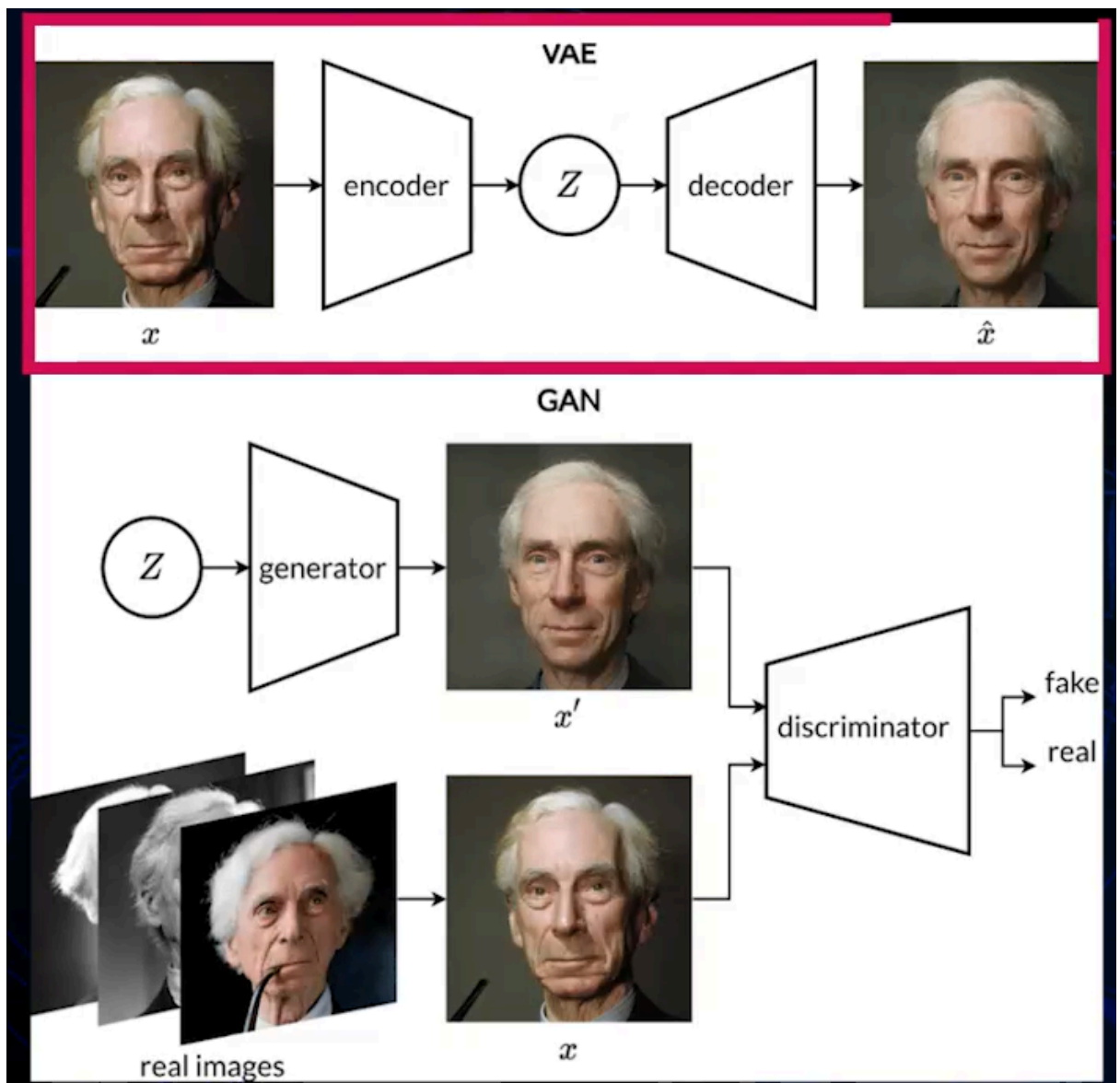
- Ferramenta de Inteligencia Artificial mostrada na aula 4
 - Gama.app
 - <https://gamma.app/pt-br>
 - Ferramenta para criar site web com IA via prompt.
- GANs Generative Adversarial Network
 - Generator
 - Discriminator



- Modelo de Aprendizado “Autoencoders Variacionais (VAE)

- https://www.google.com/search?q=exemplo+de+conte%C3%BAdo+gerado+por+um+VAS+Variational+Autoencoders&rlz=1C1GCEA_enBR1109BR1109&oq=exemplo+de+conte%C3%BAdo+gerado+por+um+VAS+Variational+Autoencoders&gs_lcrp=EgZjaHJvbWUyBggAEEUYOdIBCTE4ODczajBqN6gCCLACAfEFoxJo6MRINP7xBaMSaOjEZTT-&sourceid=chrome&ie=UTF-8#fpstate=ive&vld=cid:325afdc7,vid:CxklIRnwNvs,st:0
- https://www.google.com/search?sca_esv=bbff487b58cc43d1&rlz=1C1GCEA_enBR1109BR1109&udm=7&fbs=AlljpHxX5k-tONtMCu8aDeA7E5WMdDwGSuc8eBkl8hX51y2q6-r6qOmgvFs8yhx59bJgnXQRW0CpTUrikAvoMvruBQ5EkjQ7FupD3cXkpY3rXiGQpgEIFKuMKkMp4G-Kn6kwwutAW60X0J8LjVl1fLTlIGpnNBLcbsMn9HtOTJ1BjTkJTvk8-sKzogny1HcwPOB9WEQWBZWWhOIKEPvFTvMLeYoutJ-XRA&q=exemplo+de+conte%C3%BAdo+gerado+por+um+VAS+Variational+Autoencoders&sa=X&ved=2ahUKEwjErPWK5sePAxW3CbkGHQiONsQQtKgLegQIExAB&biw=1920&bih=911&dpr=1#fpstate=ive&vld=cid:40cd34d6,vid:Yw_05z7rt3M,st:0

○



- Ferramenta TextToVideo
- Ferramentas para IA Generativa
 - PyTorch
 - LangChain

CONTINUAR - PÁGINA 05 DE 07

*NÃO VALE NOTA. Esta é uma atividade individual. Ela não vale pontos e tem como objetivo fixar os conteúdos desenvolvidos durante a disciplina.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Dados de exemplo
textos = [
    "O novo lançamento da Apple",
    "Resultado do jogo de ontem",
    "Eleições presidenciais",
    "Atualização no mundo da tecnologia",
    "Campeonato de futebol",
    "Política internacional"
]
categorias = ["tecnologia", "esportes", "política",
"tecnologia", "esportes", "política"]

# Convertendo textos em uma matriz de contagens de tokens
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(textos)

# Dividindo os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X,
categorias, test_size=0.5, random_state=42)

# Treinando o classificador
clf = MultinomialNB()
clf.fit(X_train, y_train)

# Predição e Avaliação
y_pred = clf.predict(X_test)
print(f"Acurácia: {accuracy_score(y_test, y_pred)}")
```

Instalação do sklearn

```
pip install -U scikit-learn
```

Verificando os materiais complementares

[>>>] Keras Text to Image

keras-text-to-image - **cancelado**

Translate text to image in Keras using GAN and Word2Vec as well as recurrent neural networks

The following models are implemented in [keras_text_to_image/library]

<https://github.com/chen0040/keras-text-to-image>

[x] instalação de uma venv

[x] clone do código dentro da venv

[x] instalação via pip das dependências mapeadas no requirements.txt

[(!)] rodar os códigos

[x] pedi ajuda no discord, marquei a professora Poliana

[] tentar uma agenda com o a professora Poliana ou com alguém disposto a ajudar

Orientação da professora foi de ignorar o material

Professora Poliana via Discord.

Oii! Deixa eu perguntar, esse código você pesquisou por conta ou tem na aula?
Eu consegui fazer rodar aqui, mas os resultados não estão muito legais. Ai queria saber se realmente é pra funcionar ou não.

[] Artigo

<https://arxiv.org/abs/1406.2661>

[] GITHUB GANS

<https://github.com/diegoalejogm/gans>

[] Vídeo do Generative Adversarial Network

<https://www.youtube.com/watch?v=yz6dNf7X7SA>

[] Pipeline Generative Adversarial Network

<https://arxiv.org/pdf/1711.10742>

17/09/2025 - Iniciando a Aula de Python para ML

- Foram realizadas experimentações de diversos bilbiotecas do Python para ML
 - Numpy
 - Matplotlib
 - Pandas
 - Pytorch
 - Scikit-learn
 - Scipy
 - Seaborn
 - TensorFlow

Continuar

Aula 2 - Fundamentos para Python

<https://on.fiap.com.br/mod/conteudoshtml/view.php?id=520845&sesskey=KNWETwBmNh>

Detro do meu projeto de experimentação de Scikitlearn eu crie o programa passado pelo professor para criar um programa de Machine Learning do tipo supervisionado usando SVM - Máquina de Vetor de Suporte.

- foi usada a biblioteca LinearSVC

Suporte Vector Classifier - Classe do tipo classificador em ML supervisionada

Praticando um pouco mais de Python e ML

Realizei um exercício 17 que usa @ decoration

- Vale a penas fazer o código 18, que trata sobre Context Managers
- Também valeria a pena fazer novas versões do exemplo de Machine Learning com Scikitlearn com outros contextos de teste

<https://on.fiap.com.br/mod/conteudoshtml/view.php?id=520845&sesskey=KNWETwBmNh>

22/09/2025

Fazendo o exercício 18

```
class GerenciadorDeContexto:
    def __enter__(self):
        print("Entrando no contexto.")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Saindo do contexto.")

with GerenciadorDeContexto():
    print("Dentro do bloco with.")
```

Eu fiz um novo exercício aprofundando o uso do bloco “with” para entender a ordem de execução do bloco e dos início e finalização dos contextos `__enter__` e `__exit__`

o uso do bloco `with` em Python, que é uma maneira conveniente de gerenciar recursos externos, como arquivos ou conexões de rede, garantindo que eles sejam limpos corretamente, mesmo que ocorram erros.

O bloco `with` no Python é uma ferramenta poderosa para gerenciar recursos de forma segura e eficiente, garantindo que eles sejam liberados corretamente após o uso, mesmo em caso de erros.

Ele funciona em conjunto com objetos que implementam o **protocolo de gerenciamento de contexto**, que consiste em dois métodos especiais: `__enter__` e `__exit__`.

```
class Cadastro:
    def __init__(self, nome, telefone):
        print("\n1 - Entrando no construtor...")
        self.nome = nome
        self.telefone = telefone
        print(f"Nome: {self.nome} - Telefone: {self.telefone} ")
        input()

    def __enter__(self):
        print("\n2 - Entrando no contexto da classe...")
        print(f"Nome: {self.nome} - Telefone: {self.telefone} ")
        input()

    def __exit__(self, exc_type, exc_value, traceback):
        print("\n4 - Saindo do Contexto da classe...")
        print(f"Nome: {self.nome} - Telefone: {self.telefone} ")
        input()

print("\n0 - agora iniciará um bloco 'with Cadastro' ")
input()
with Cadastro("Bruno", "9999"):
    print("\n3 - Dentro do bloco with")
    print("Aqui não tenho acesso ao atributo telefone")
    input()

# Exemplo de uso:
# Criando um objeto da classe Cadastro
print("\n5 - instanciando um objeto da classe Cadastro")
```

```
input()
pessoal = Cadastro("João", "1111")

# Acessando os atributos do objeto
print("\n6 - imprimindo dos dados do objeto instanciado")
input()
print(f"Nome: {pessoal.nome}")
print(f"Telefone: {pessoal.telefone}")
```

List Comprehension

Exercício

```
'''
List Comprehension

É uma forma de criar uma lista a partir de um for interno que pode
compreender um operação sobre os valores

Exemplo desse exercício é criar uma lista de 10 números. Cada número
será o número da iteração do loop elevado ao quadrado
'''

lista_quadrados = [x**2 for x in range(10)]
print(lista_quadrados)
```

Aprofundamento no assunto

https://www.hashtagtreinamentos.com/list-comprehension-python?gad_source=1&gclid=CjwKCAjwps-zBhAiEiwALwsVYQ0JYnnNNinmVCZzMbCdHHKH_0opM3a3JvDJizHGJ4BTNdFI613U-hoCRkUQAvD_BwE

Mais um exercício com List Comprehension com condicional embutido

```
'''
List Comprehension

É uma forma de criar uma lista a partir de um for interno que pode
compreender uma operação sobre os valores

Exemplo desse exercício é criar uma lista de 10 números. Cada número
será o número da iteração do loop elevado ao quadrado
'''

lista = [10,20,30,40,50,60]

lista2 = [numero * 2 for numero in lista]

maiores_que_100 = [numero for numero in lista2 if numero > 100]

print(f"\n lista original: {lista}")
print(f"\n lista dobrada: {lista2}")
print(f"\n lista dos números maiores que 100: {maiores_que_100}\n")
```

24/09/2025

Criação de módulos e bibliotecas

<https://on.fiap.com.br/mod/conteudoshtml/view.php?id=520846&sesskey=3CHaUyD6sD>

Gerenciador de pacotes do Python

- Pypi - <https://pypi.org/>
 - Repositório do instrutor
 - pypi.org/user/tadrianonet/
 - Criando a minha conta
 - https://docs.google.com/document/d/1NVRO3HBs8CeEPbiSMOW_KGk6yDz4-50e2FRrUJFzh1l/edit?tab=t.0
 - Exemplo de um componente interessante para baixar e testar
 - python-pptx 1.0.2
 - https://docs.google.com/document/d/1NVRO3HBs8CeEPbiSMOW_KGk6yDz4-50e2FRrUJFzh1l/edit?tab=t.0
 -

- Como criar um pacote para envio para o Pypi
 - Estrutura
 - O instrutor falou num script create_structure.sh
 - Esse script cria toda a estrutura para criar o pacote
 - ./create_structure.sh cursofiap

```
./create_structure.sh: line 61: # ${STRUCTURE_NAME^}

A simple example library.

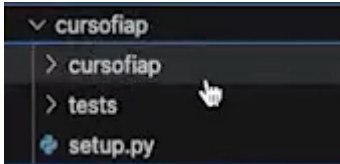
## Installation

\`\`\`sh
pip install ${STRUCTURE_NAME}
\`\`\`

## Usage

\`\`\`python
from ${STRUCTURE_NAME} import hello_world

print(hello_world())
\`\`\`
: bad substitution
Estrutura cursofiap criada com sucesso!
○ (env) (base) thiagoadriano@Thiagos-MBP tools %
```

- - 

- Faltou o [README.md](#)
 - Criar esse manualmente

Nessa aula foi disponibilizado todo o código de uma aplicação de ações.

Eu obti em não praticar porque estou fazendo um curso paralelo em Python e então passei direto disso.

- Na aula seguinte a abordagem foi sobre API, faltando de FastAPI e Flask
- Eu já fiz muitos exemplos com FastAPI
- Já o Flask fiz um pequeno exemplo e vou iniciar em breve um curso na Alura de Python Flask e ChatGPT.
- Então eu assisti a aula mas não fiz o exercício

25/09/2025

Frameworks de ML em Python

- No primeiro vídeo: Foram explorado diversos exemplos de forma muito superficial, como PyTorch, Keras, TensorFlow e Scikit-learn
- No segundo vídeo: Foi abordado **LangChain** que é um conhecimento técnico novo pra mim.

LangChain

LangChain é um **framework de código aberto** (conjunto de ferramentas e bibliotecas) projetado para **simplificar o desenvolvimento de aplicações** que utilizam **Grandes Modelos de Linguagem (LLMs)**, como GPT, Bard e outros.

Em essência, ele funciona como um orquestrador que permite "encadear" (daí o nome *Chain*) diferentes componentes para criar fluxos de trabalho complexos e inteligentes.

O Propósito Principal

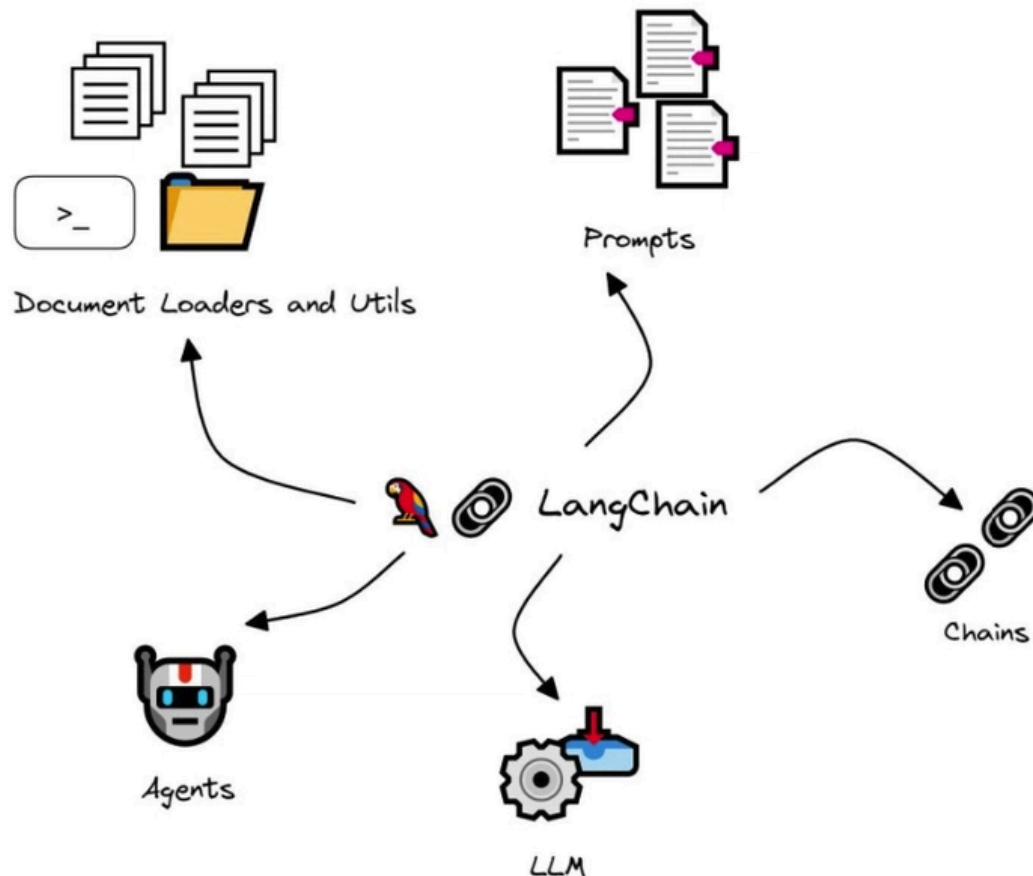
O LangChain resolve uma limitação chave dos LLMs: eles são excelentes em gerar texto, mas, por padrão, não têm acesso a informações em **tempo real** ou a dados **específicos e proprietários** (como documentos internos de uma empresa, e-mails, ou o resultado de uma pesquisa na web no momento).

Ele permite que você:

1. **Conecte LLMs a fontes de dados externas:** Isso pode incluir documentos, bancos de dados, APIs, a web, etc.
2. **Dê "Memória" aos LLMs:** Permite que os modelos se lembrem do contexto de conversas anteriores.
3. **Habilite Ações:** Permite que os LLMs usem **ferramentas** (como uma calculadora, uma API de pesquisa, ou um código Python) para realizar tarefas, tornando-os **agentes** mais poderosos.

<https://dev.to/programadriano/conhecendo-o-langchain-egk>

- Criação e gerenciamento de Pipelines de processamento de linguagem natural



Toolkit da LangChain

- **Document Loaders and Utils**
 - Carregadores de documentos e utilitários;
 - Ler processar e carregar diversas fontes de arquivos
 - Texto;
 - PDFs;
 - Banco de Dados;
 - Para serem usados em operações de processamento de linguagem natural;
- **Prompts**
 - Instruções ou perguntas fornecidas ao modelo de linguagem para gerar respostas ou realizar uma tarefa específica;
 - Eles são fundamentais para direcionar o comportamento do modelo de linguagem natural;
- **Chains (ou cadeias)**
 - Elas representam uma sequência de etapas ou operações que são realizadas em conjunto para alcançar um objetivo específico;
 - É possível encadear LLMs para aplicações mais complexas, entre si ou com outros módulos especializados;
- **Indexes (Índices):**
 - Mecanismos para interagir com dados externos. O mais famoso é o **Retrieval-Augmented Generation (RAG)**, que permite buscar informações relevantes em sua base de dados antes de alimentar o LLM com o *prompt*.

- **Memory (Memória):**
 - Armazena o histórico da conversa para dar contexto ao LLM.
- **Agents**
 - Eles são componentes que agem de forma autônoma para realizar tarefas específicas baseadas em entradas e em modelos de linguagem. Eles podem tomar decisões e realizar ações com base em regras ou aprendizagem.

Aplicações Comuns

LangChain é usado para criar aplicações sofisticadas de IA, como:

- **Chatbots Inteligentes:** Que mantêm o contexto da conversa e podem acessar informações externas.
- **Sistemas de Perguntas e Respostas (RAG):** Capazes de responder perguntas com base em um conjunto específico de documentos (por exemplo, um manual técnico ou relatórios da empresa).
- **Análise e Resumo de Documentos:** Ferramentas que podem processar grandes volumes de texto e gerar resumos concisos.
- **Agentes Autônomos:** Que podem executar uma série de etapas e usar múltiplas ferramentas para realizar tarefas complexas.

Em resumo, **LangChain é uma ponte** que leva os LLMs de meros geradores de texto a serem o motor de raciocínio de **aplicações de IA completas e conectadas** ao mundo real e a dados específicos.

Baixar e instalar todos os exemplos do professor

<https://github.com/FIAP/MLET-PYTHON-LIBS-IA/tree/main/exemplo5.1>

[OK] Testando o exemplo1 da Aula 5

- Esse algoritmo implementa um fluxo básico de aprendizagem de máquina para prever um valor (“preco_fechamento”) com base em uma data, utilizando um modelo **Random Forest Regressor**.

[OK] Criando um código meu para prever o preço de um carro especificamente

Ficou excelente, inclusive aprendi a listar as predição com x elementos do topo ou x elemento do fim do conjunto ordenando pelo preço ou ano de forma crescente ou decrescente

Random Forest Regressor

O Random Forest Regressor é um algoritmo de aprendizado de máquina (Machine Learning) muito popular e eficaz, usado para tarefas de regressão.

Ele faz parte da família dos algoritmos baseados em árvores de decisão e é um tipo de modelo de conjunto (**ensemble method**), especificamente um método de **bagging** (do inglês, **bootstrap aggregation**).

Como o Random Forest Regressor Funciona?

1. Floresta de Árvores

O nome "Floresta Aleatória" (**Random Forest**) vem do seu funcionamento. Em vez de construir apenas uma árvore de decisão, **ele constrói uma grande coleção (floresta) delas durante o treinamento**.

2. Aleatoriedade (**Randomness**)

A "aleatoriedade" é introduzida em dois pontos principais para garantir que as árvores não sejam idênticas:

- Amostragem de Dados (**Bagging**): Cada árvore é treinada em uma amostra diferente do conjunto de dados original, tirada com reposição (**bootstrap sample**). Isso significa que alguns pontos de dados podem ser usados várias vezes e outros podem não ser usados.
- Seleção de Features (**Características**): Ao construir cada árvore, em cada nó, **o algoritmo considera apenas um subconjunto aleatório das características disponíveis para encontrar a melhor divisão**. Isso força as árvores a se concentrarem em diferentes *features* e evita que um punhado de *features* dominantes domine a construção de todas as árvores.

3. Previsão (**Prediction**)

Quando você quer fazer uma previsão com o Random Forest Regressor:

- Ele passa o novo dado de entrada por todas as árvores na floresta.
 - Cada árvore retorna sua própria previsão de valor (que é um número real, pois é regressão).
 - **O algoritmo, então, calcula a média de todas as previsões das árvores individuais para obter a previsão final.**
-

Por Que Ele é Tão Popular?

- **Alta Precisão:** A média das previsões de várias árvores (que tendem a compensar os erros umas das outras) geralmente resulta em uma previsão final mais robusta e precisa do que qualquer árvore individual.
 - **Redução de Overfitting:** A aleatoriedade no processo de construção de árvores e o fato de serem modelos de conjunto ajudam a mitigar o overfitting (o modelo se ajustar *demais* aos dados de treinamento, perdendo a capacidade de generalizar para novos dados), um problema comum em árvores de decisão únicas.
 - **Fácil Interpretação e Configurar:** Embora a floresta em si seja complexa, o algoritmo requer relativamente poucos hiperparâmetros para ajuste e não exige uma grande preparação dos dados (como normalização). Ele também fornece uma medida útil de importância de características, mostrando quais variáveis foram mais cruciais para as previsões.
-

Exemplo de Uso

O Random Forest Regressor é usado em cenários onde o objetivo é prever um valor contínuo (numérico), como:

- Previsão de preços: Estimar o preço de uma casa ou de uma ação.
- Previsão de demanda: Estimar a quantidade de produto que será vendida.
- Previsão de risco: Estimar a probabilidade de um evento, como a falha de um equipamento.

Explicação feita pela Gemini do exercício 1 - predição de valor da ação

Esse código em Python implementa um fluxo básico de aprendizagem de máquina para prever um valor ('preco_fechamento') com base em uma data, utilizando um modelo Random Forest Regressor.

1. Explicação das Bibliotecas

O código começa importando cinco bibliotecas essenciais para tarefas de análise de dados e machine learning:

Biblioteca	Alias Comum	Função Principal
<code>pandas</code>	<code>pd</code>	Utilizada para manipulação e análise de dados, fornecendo estruturas de dados de alto desempenho como DataFrames. É a principal ferramenta para carregar, limpar e preparar os dados.
<code>sklearn.model_selection</code>	-	Fornece utilitários para dividir conjuntos de dados, incluindo a função <code>train_test_split</code> , essencial para separar dados de treino e teste de forma correta.
<code>sklearn.ensemble</code>	-	Contém algoritmos de <i>ensemble</i> (conjuntos de modelos), como o RandomForestRegressor, um poderoso modelo de aprendizado de máquina usado para tarefas de regressão (previsão de valores contínuos).
<code>sklearn.metrics</code>	-	Oferece ferramentas para avaliar a performance do modelo, como <code>mean_squared_error</code> , que calcula o erro médio quadrático entre as previsões e os valores reais.
<code>numpy</code>	<code>np</code>	Uma biblioteca fundamental para computação numérica em Python. É usada para trabalhar com arrays (vetores e matrizes) e realizar operações matemáticas de alto nível, como calcular a raiz quadrada (<code>np.sqrt</code>).

2. Explicação Passo a Passo do Código

O código executa uma sequência de 8 passos principais, desde o carregamento dos dados até a avaliação do modelo:

Passo 1: Carregamento dos Dados

Python

```
# Carregando os dados a partir de um arquivo JSON
```

```
df = pd.read_json('data.json')
```

- A função `pd.read_json()` (do pandas) é usada para ler os dados contidos em um arquivo chamado `data.json` e carregá-los em um `DataFrame` do pandas, que é armazenado na variável `df`.

Passo 2: Pré-processamento da Coluna de Data

Python

```
# Convertendo a coluna de data para o tipo datetime
```

```
df['data'] = pd.to_datetime(df['data'])
```

- A coluna `data` do `DataFrame` é convertida para o tipo de dado `datetime` do pandas. Isso é crucial para que o pandas possa manipular e extrair informações de data e hora de forma eficiente.

Passo 3: Engenharia de Recursos (Feature Engineering)

Python

```
# Extraindo recursos (features) e o alvo (target)
```

```
df['data_ordinal'] = df['data'].map(pd.Timestamp.toordinal)
```

```
X = df[['data_ordinal']]
```

```
y = df['preco_fechamento']
```

1. Criação de Recurso: O modelo de regressão não pode processar datas diretamente no formato `datetime`. A função `.toordinal()` é aplicada à coluna `data` para convertê-la em um número inteiro que representa a contagem de dias desde uma data de referência (1º de Janeiro do ano 1). Esse novo recurso numérico é salvo na coluna `data_ordinal`.
2. Definição de Variáveis:
 - `X` (Variáveis Independentes/Features): É o recurso que será usado para a previsão. Nesse caso, é a coluna `data_ordinal` (como um `DataFrame`).
 - `y` (Variável Dependente/Target): É o valor que o modelo tentará prever, a coluna `preco_fechamento` (como uma `Série/Series` do pandas).

Passo 4: Divisão dos Dados em Treino e Teste

Python

```
# Dividindo os dados em conjuntos de treinamento e teste
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- A função `train_test_split()` divide os conjuntos de dados `X` e `y` em quatro subconjuntos:
 - `X_train` e `y_train`: Usados para treinar o modelo.
 - `X_test` e `y_test`: Usados para avaliar o modelo em dados que ele nunca viu.
- `test_size=0.2` indica que 20% dos dados serão usados para o conjunto de teste e 80% para o treino.

- `random_state=42` garante que a divisão dos dados seja a mesma toda vez que o código for executado (reprodutibilidade).

Passo 5: Treinamento do Modelo

Python

```
# Treinando o modelo RandomForestRegressor
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

1. Inicialização: Uma instância do modelo `RandomForestRegressor` é criada e armazenada na variável `model`. O parâmetro `n_estimators=100` significa que o modelo usará 100 árvores de decisão (aumentando a robustez) e `random_state=42` garante a reprodutibilidade.
2. Treinamento (`.fit()`): O método `.fit()` é chamado, passando os dados de treino (`X_train, y_train`). Este é o momento em que o modelo aprende a relação entre as datas ordinais e os preços de fechamento.

Passo 6: Previsão

Python

```
# Fazendo previsões
y_pred = model.predict(X_test)
```

- O método `.predict()` é chamado no modelo treinado, usando os dados de teste (`X_test`).
- O modelo aplica o que aprendeu para gerar previsões de preço, que são armazenadas na variável `y_pred`.

Passo 7: Avaliação do Modelo

Python

```
# Avaliando o modelo
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
print(f"Root Mean Squared Error: {rmse}")
```

1. Cálculo do Erro Médio Quadrático (MSE): A função `mean_squared_error()` (do `sklearn.metrics`) calcula a média das diferenças quadráticas entre os preços reais (`y_test`) e os preços previstos (`y_pred`).
2. Cálculo do Raiz do Erro Médio Quadrático (RMSE): O `np.sqrt()` (do `numpy`) é usado para calcular a raiz quadrada do MSE, resultando no RMSE. O RMSE é a métrica mais comum em regressão e tem a vantagem de estar na mesma unidade da variável alvo (o preço), facilitando a interpretação do erro médio do modelo.
3. O valor do RMSE é então impresso.

Passo 8: Exibição das Previsões

Python

```
# Exibindo algumas previsões
predictions = pd.DataFrame({'Data': X_test['data_ordinal'].map(lambda x:
pd.Timestamp.fromordinal(x)), 'Real': y_test, 'Predito': y_pred})
print(predictions.head())
```

1. Criação do DataFrame de Resultados: Um novo DataFrame chamado predictions é criado para comparar as previsões e os valores reais de forma organizada.
2. Conversão de Volta: Para tornar o resultado legível, a coluna data_ordinal (do X_test) é convertida de volta ao formato de data (pd.Timestamp.fromordinal()).
3. Exibição: O método .head() é usado para exibir as primeiras 5 linhas desse DataFrame, mostrando lado a lado a Data, o preço Real e o preço Predito pelo modelo.

Antes de seguir para o exercício 2, cabe uma indagação:

Como eu faço para prever o valor médio do carro em 2040 se meu dataset (.json) só tem dados até 2033?

Você tem uma excelente pergunta, que toca em um ponto crucial da aplicação de modelos de Machine Learning: **a extrapolação para dados fora do intervalo de treinamento.**

O modelo **Random Forest Regressor** (e a maioria dos modelos baseados em árvores de decisão, como **DecisionTreeRegressor** e **GradientBoostingRegressor**) tem uma limitação fundamental quando se trata de prever valores para anos muito além do seu conjunto de dados de treinamento (que vai até 2033 no seu caso).

A Limitação do Random Forest para Extrapolação:

Um modelo Random Forest só pode prever valores que estão dentro do intervalo de valores-alvo (y) que ele viu durante o treinamento. Ele não pode "inventar" um valor que seja maior ou menor do que o máximo ou mínimo valor de preço do carro no seu conjunto de treinamento. De forma simplificada, ele prevê a média dos valores de treinamento associados a uma folha da árvore de decisão, mas ele não tem a capacidade de prever uma tendência contínua para fora dos dados que aprendeu.

Para fazer uma previsão para um ano como 2040, você precisa de um algoritmo que seja capaz de extrapolação, ou seja, que possa prever valores fora do intervalo conhecido.

Como Fazer uma Predição para 2040 (**Extrapolação**)

Para obter uma previsão para 2040, você precisa realizar duas etapas principais:

1. Preparar os Dados de Entrada para 2040

O modelo precisa do ano no mesmo formato que foi treinado, ou seja, o Ordinal do Timestamp.

Você pode criar a entrada para o ano 2040 da seguinte forma:

Python

1. Definir o ano futuro para a predição

```
ano_futuro = pd.to_datetime('2040-02-20') # Escolha um dia, o mês/dia não importam muito
ano_futuro_ordinal = ano_futuro.toordinal()
```

2. Criar o DataFrame de entrada com o formato que o modelo espera (uma matriz/coluna)

O Random Forest foi treinado com um DataFrame de uma coluna, então a entrada deve ser um array bidimensional

```
X_futuro = np.array([[ano_futuro_ordinal]])
```

2. Escolher um Modelo de Regressão que Suporta Extrapolação

Para uma **extrapolação linear no tempo**, o modelo mais simples e eficaz seria a **Regressão Linear**.

Substitua o `RandomForestRegressor` por um modelo de regressão linear.

A Regressão Linear irá aprender a relação linear entre o tempo (`ano_ordinal`) e o preço (`preco_medio`) e pode facilmente estender essa linha para o futuro.

Abaixo está o código modificado usando a Regressão Linear, que é o mais indicado para este tipo de extrapolação:

Python

```
# Importar o modelo de Regressão Linear
```

```
from sklearn.linear_model import LinearRegression
```

```
# ... (manter os outros imports)
```

```
# ... (código de carregamento e preparação dos dados é o mesmo)
```

```
df =
```

```
pd.read_json('/home/brunojose/devops/python/fiap-ia-devs-aula5/exemplo1/precos_lr_fl2_sdk_se_2013.json')
```

```
df['ano'] = pd.to_datetime(df['ano'])
```

```
df['ano_ordinal'] = df['ano'].map(pd.Timestamp.toordinal)
```

```

X = df[['ano_ordinal']]
y = df['preco_medio']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5,
random_state=50)

# Treinando o modelo agora com Regressão Linear
# model = RandomForestRegressor(n_estimators=200, random_state=50) <-- REMOVER
model = LinearRegression() # <-- USAR ESTE MODELO
model.fit(X_train, y_train)

# ... (a avaliação e as previsões de teste são as mesmas)
# O RMSE será muito diferente, mas o objetivo é a capacidade de extrapolação.

# -----
# PASSO NOVO: Predição para o ano 2040
# -----

# 1. Definir o ano futuro (2040)
ano_predicao = 2040
data_futura = pd.to_datetime(f'{ano_predicao}-02-20')
ano_futuro_ordinal = data_futura.toordinal()

# 2. Preparar a entrada para o modelo (array 2D)
X_futuro = np.array([[ano_futuro_ordinal]])

# 3. Fazer a predição
preco_2040_predito = model.predict(X_futuro)[0]

print(f"\n--- Predição para {ano_predicao} ---")
print(f"Preço médio predito para o ano {ano_predicao}: R$ {preco_2040_predito:.2f}")

```

Resumo da Solução

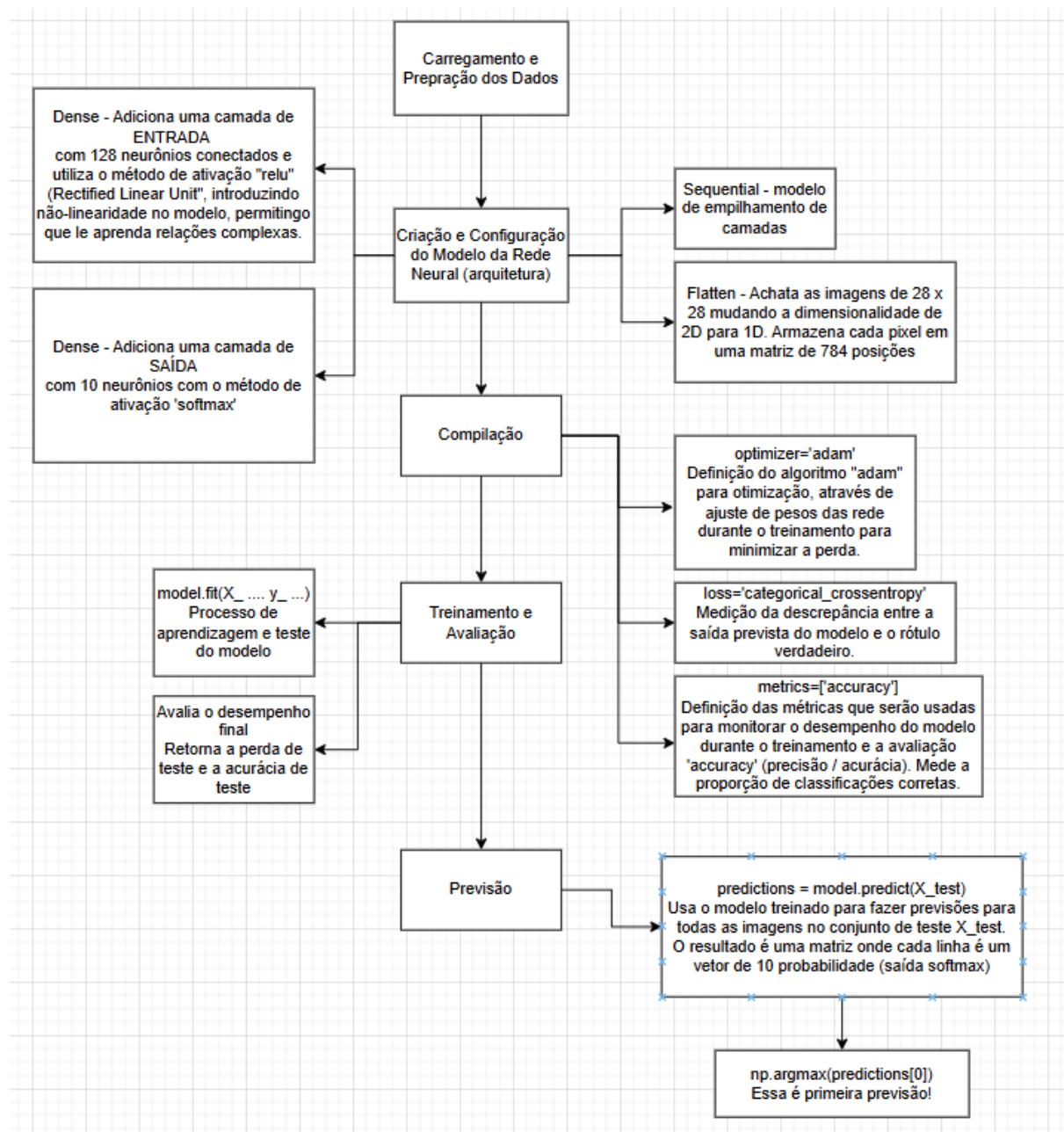
1. Mude o Algoritmo: Use a Regressão Linear (`LinearRegression`) ou outro modelo que possa extrapolar (como modelos de séries temporais mais avançados, como ARIMA, se a ordem dos dados for importante) em vez do Random Forest.
2. Crie a Entrada de Teste: Converta a data futura (2040-02-20) para o formato Ordinal que o modelo espera.
3. Preveja: Use o método `.predict()` do novo modelo com a entrada de 2040.

Eu aproveitei e fiz também um programa para calcular o valor do m2 em águas claras em qualquer ano.

Eu implementei o código e ficou muito bom!

27/09/2025

CONTINUAR NO EXERCÍCIO 2



Explicação do passo a passo pelo Gemini IA

Olá! Com certeza, vou explicar cada comando desse código Python que utiliza a biblioteca TensorFlow/Keras para criar, treinar e avaliar um modelo de Rede Neural Artificial simples para

classificação de dígitos escritos à mão do conjunto de dados MNIST.

Importação de Bibliotecas

Esta seção carrega as ferramentas necessárias para o código:

Comando	Descrição
<code>from tensorflow.keras.datasets import mnist</code>	Importa o módulo <code>mnist</code> , que contém a função para carregar o famoso conjunto de dados MNIST de imagens de dígitos.
<code>from tensorflow.keras.models import Sequential</code>	Importa a classe <code>Sequential</code> , que é a maneira mais simples de construir um modelo Keras, onde as camadas são empilhadas sequencialmente.
<code>from tensorflow.keras.layers import Dense, Flatten</code>	Importa as classes de camadas: <code>Dense</code> (camada totalmente conectada, onde cada neurônio se conecta a todos os neurônios da camada anterior) e <code>Flatten</code> (usada para "achatar" a entrada, transformando uma matriz 2D de pixels em um vetor 1D).
<code>from tensorflow.keras.utils import to_categorical</code>	Importa a função <code>to_categorical</code> , que é usada para converter rótulos inteiros em vetores de codificação <i>one-hot</i> .
<code>import numpy as np</code>	Importa a biblioteca <code>numpy</code> e a renomeia como <code>np</code> , que é essencial para operações numéricas e manipulação de arrays (vetores/matrizes), que é como os dados são armazenados.

Carregamento e Preparação de Dados

Esta seção baixa e prepara os dados para que o modelo possa usá-los.

Comando	Descrição

<code>(X_train, y_train), (X_test, y_test) = mnist.load_data()</code>	Baixa e carrega o conjunto de dados MNIST. Ele divide os dados em: <code>X_train</code> (imagens de treinamento), <code>y_train</code> (rótulos de treinamento), <code>X_test</code> (imagens de teste) e <code>y_test</code> (rótulos de teste).
<code>X_train = X_train / 255.0</code> <code>X_test = X_test / 255.0</code>	Normaliza os dados de imagem. As imagens MNIST têm valores de pixel de 0 a 255. A divisão por 255.0 escala esses valores para o intervalo de 0 a 1, o que geralmente ajuda a acelerar e estabilizar o treinamento da rede neural.
<code>y_train = to_categorical(y_train, 10)</code> <code>y_test = to_categorical(y_test, 10)</code>	Converte os rótulos (os números 0 a 9) para o formato <i>one-hot encoding</i> . Por exemplo, o dígito 3 é convertido em um vetor <code>[0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]</code> . O segundo argumento, <code>10</code> , indica que existem 10 classes possíveis (os dígitos 0 a 9).

Criação e Configuração do Modelo

Aqui, a arquitetura da rede neural é definida e configurada para o treinamento.

Criação da Arquitetura

Comando	Descrição
<code>model = Sequential([...])</code>	Inicia a criação de um modelo sequencial, onde você define as camadas que se conectam da entrada à saída.
<code>Flatten(input_shape=(28, 28))</code>	Adiciona a primeira camada. A camada <code>Flatten</code> (Achatar) pega cada imagem de 28×28 pixels e a transforma em um vetor linear de 784 (28×28) elementos. O <code>input_shape</code> define o formato das imagens de entrada.
<code>Dense(128, activation='relu')</code>	Adiciona uma camada <code>Dense</code> (totalmente conectada) com 128 neurônios. A função de ativação <code>'relu'</code> (Rectified Linear Unit) introduz não-linearidade no modelo, permitindo que ele aprenda relações complexas.

<code>Dense(10, activation='softmax')</code>	Adiciona a camada de saída. Tem 10 neurônios, um para cada classe de dígito (0 a 9). A função de ativação 'softmax' garante que as saídas (probabilidades) somem 1, fornecendo a probabilidade de a imagem pertencer a cada uma das 10 classes.
--	---

Compilação

Comando	Descrição
<code>model.compile(...)</code>	Configura o processo de aprendizado do modelo.
<code>optimizer='adam'</code>	Define o algoritmo de otimização que ajustará os pesos da rede durante o treinamento para minimizar a perda. 'adam' é uma escolha popular e eficiente.
<code>loss='categorical_crossentropy'</code>	Define a função de perda (loss), que mede a discrepância entre a saída prevista do modelo e o rótulo verdadeiro. 'categorical_crossentropy' é a função de perda padrão para problemas de classificação multi-classe com rótulos <i>one-hot</i> .
<code>metrics=['accuracy']</code>	Define as métricas que serão usadas para monitorar o desempenho do modelo durante o treinamento e a avaliação. 'accuracy' (precisão/acurácia) mede a proporção de classificações corretas.

Treinamento e Avaliação

Estes comandos executam o processo de aprendizado e teste do modelo.

Comando	Descrição

<code>model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.2)</code>	Treina o modelo usando os dados de treinamento (<code>X_train</code> e <code>y_train</code>). - <code>epochs=5</code> : O modelo fará 5 passagens completas por todo o conjunto de treinamento. - <code>batch_size=32</code> : A rede processará 32 amostras de cada vez antes de atualizar os pesos. - <code>validation_split=0.2</code> : 20% dos dados de treinamento serão reservados para monitorar o desempenho do modelo em dados não vistos durante cada época (dados de validação).
<code>test_loss, test_acc = model.evaluate(X_test, y_test)</code>	Avalia o desempenho final do modelo nos dados de teste (<code>X_test</code> e <code>y_test</code>), que são totalmente novos para o modelo. Ele retorna a perda de teste (<code>test_loss</code>) e a acurácia de teste (<code>test_acc</code>).
<code>print(f'Test accuracy: {test_acc}')</code>	Exibe a acurácia final do modelo no conjunto de teste.

Previsão

Esta seção demonstra como usar o modelo treinado para fazer previsões em novos dados.

Comando	Descrição
<code>predictions = model.predict(X_test)</code>	Usa o modelo treinado para fazer previsões para todas as imagens no conjunto de teste (<code>X_test</code>). O resultado é uma matriz onde cada linha é um vetor de 10 probabilidades (saída <code>softmax</code>).
<code>print(f'Predicted label: {np.argmax(predictions[0])}')</code>	Pega a primeira previsão (<code>predictions[0]</code>) e usa <code>np.argmax()</code> para encontrar o índice (que é o dígito previsto) da maior probabilidade nesse vetor de 10 probabilidades.
<code>print(f'True label: {np.argmax(y_test[0])}')</code>	Pega o primeiro rótulo real (<code>y_test[0]</code>), que está em formato <i>one-hot</i> , e usa <code>np.argmax()</code> para convertê-lo de volta para o dígito original (o rótulo verdadeiro).

Espero que esta explicação detalhada tenha sido clara! Se tiver mais dúvidas sobre algum

conceito, é só perguntar.

01/10/2025

Exercício 3

Explicações da IA Gemini

Excelente! Esse código demonstra um exemplo básico e fundamental de como **treinar** um modelo de regressão linear simples usando a biblioteca **PyTorch**.

Vou explicar cada parte passo a passo:

1\ Importações

```
```python
import torch
import torch.optim as optim
```
```

* **import torch** *: Importa a biblioteca principal **PyTorch**, essencial para a criação de tensores (estruturas de dados), modelos e operações de cálculo.

* **import torch.optim as optim** *: Importa o módulo `optim` do PyTorch, que contém algoritmos de otimização como o **SGD** (Stochastic Gradient Descent), usado para ajustar os parâmetros do modelo.

2\ Dados de Entrada e Saída

```
```python
Dados de entrada e saída desejada
x_data = torch.tensor([[1.0], [2.0], [3.0], [4.0]], requires_grad=False)
y_data = torch.tensor([[2.5], [3.5], [5.5], [6.5]], requires_grad=False)
```
```

* **x_data** *: É o **tensor de entrada** (variável independente, ou *features*). Ele tem 4 exemplos, cada um com 1 característica (o valor é 1.0, 2.0, 3.0, 4.0). O formato é

[número de exemplos, número de características]`.
 * **`y\_data`**: É o **tensor de saída desejada** (rótulos, ou *targets*). Ele tem os valores que o modelo deve aprender a prever.
 * **`requires\_grad=False`**: Indica que o PyTorch **não** precisa calcular os gradientes (derivadas) para esses tensores durante o treinamento, pois eles são apenas dados de entrada/saída, e não parâmetros ajustáveis do modelo.

3\. Definição do Modelo de Rede Neural

```
```python
Definindo um modelo de rede neural simples com uma camada linear
class SimpleNN(torch.nn.Module):
 def __init__(self):
 super(SimpleNN, self).__init__()
 self.linear = torch.nn.Linear(1, 1) # Uma camada linear

 def forward(self, x):
 return self.linear(x)
```
```

* **`class SimpleNN(torch.nn.Module)`**: Define uma classe para o nosso modelo. Todo modelo de rede neural no PyTorch deve herdar de `torch.nn.Module`.

* **`def \_\_init\_\_(self)`**: O construtor da classe.

* **`super(SimpleNN, self).\_\_init\_\_()`**: Chama o construtor da classe pai (`nn.Module`).

* **`self.linear = torch.nn.Linear(1, 1)`**: Cria uma **camada linear** (ou camada densa).

- * O primeiro `1` indica que a entrada tem 1 característica.
- * O segundo `1` indica que a saída tem 1 característica.
- * Esta camada implementa a função $\hat{y} = w \cdot x + b$, onde w (peso) e b (bias/viés) são os parâmetros que serão aprendidos.

* **`def forward(self, x)`**: Define o **passo à frente (forward pass)** do modelo. É a função que será chamada para fazer previsões.

* **`return self.linear(x)`**: Simplesmente passa a entrada `x` através da camada linear e retorna a saída.

4\. Inicialização de Componentes

```
```python
Inicializando o modelo
model = SimpleNN()

Função de perda
loss_function = torch.nn.MSELoss()
```

```
Configurando o otimizador
optimizer = optim.SGD(model.parameters(), lr=0.01)
'''

* **`model = SimpleNN()`** : Cria uma instância do nosso modelo.
* **`loss_function = torch.nn.MSELoss()`** : Define a Função de Perda (ou Loss Function).
 * **`MSELoss`** (Mean Squared Error - Erro Quadrático Médio) é uma escolha comum para problemas de regressão. Ela mede o "quão errado" o modelo está, calculando a média do quadrado das diferenças entre as previsões ($\hat{y}$) e os valores reais (y).
* **`optimizer = optim.SGD(model.parameters(), lr=0.01)`** : Configura o Otimizador.
 * **`optim.SGD`** (Stochastic Gradient Descent) é o algoritmo que será usado para ajustar os pesos e biases do modelo.
 * **`model.parameters()`** : Diz ao otimizador quais parâmetros ele deve ajustar (os pesos e biases da camada linear).
 * **`lr=0.01`** : Define a taxa de aprendizado (learning rate). É o tamanho do "passo" que o otimizador dará na direção do gradiente para minimizar a perda.
```

-----

## ## 5\ Loop de Treinamento

```
```python
# Treinamento do modelo
for epoch in range(100):
    # Forward pass
    y_pred = model(x_data)

    # Cálculo da perda
    loss = loss_function(y_pred, y_data)

    # Retropropagação e otimização
    optimizer.zero_grad() # Zera os gradientes acumulados
    loss.backward()       # Calcula gradientes
    optimizer.step()       # Atualiza parâmetros
'''
```

Este é o **coração** do treinamento, repetido 100 vezes (**epochs**):

1. **`y\_pred = model(x\_data)` (Forward Pass)**: O modelo recebe os dados de entrada (`x_data`) e faz suas previsões (`y_pred`).
2. **`loss = loss\_function(y\_pred, y\_data)` (Cálculo da Perda)**: A perda é calculada comparando as previsões (`y_pred`) com os valores reais (`y_data`).
3. **`optimizer.zero\_grad()`**: **Crucial.** Por padrão, o PyTorch acumula gradientes em cada iteração. É preciso zerar os gradientes acumulados da iteração anterior antes de calcular os novos.
4. **`loss.backward()` (Retropropagação / Backpropagation)**: Calcula os **gradientes** (derivadas) da função de perda em relação a **todos** os parâmetros ajustáveis do modelo. Isso informa a direção e a magnitude do ajuste necessário.
5. **`optimizer.step()` (Otimização)**: O otimizador usa os gradientes recém-calculados

(do passo 4) e a taxa de aprendizado (`lr`) para **atualizar** os pesos e biases do modelo. Esta é a etapa de **aprendizado**.

6\ Resultados e Previsão Final

```
```python
Exibindo parâmetros finais
for name, param in model.named_parameters():
 print(f'{name}: {param.data.item()}')

Fazer uma previsão com o modelo treinado
x_test = torch.tensor([[5.0]])
y_test_pred = model(x_test)
print(f'Previsão para x=5.0: {y_test_pred.item()}')
```
```

*****`model.named_parameters()`***:** Itera sobre os parâmetros finais aprendidos pelo modelo (o peso `$w$` e o bias `$b$`).

- * O nome será algo como ``linear.weight`` e ``linear.bias``.
- * ``param.data.item()`` extrai o valor numérico (aprendido) do tensor.

*****`x_test = torch.tensor([[5.0]])`***:** Cria um novo tensor de entrada para teste.

*****`y_test_pred = model(x_test)`***:** O modelo, agora treinado, é usado para fazer uma previsão para a entrada ``5.0``.

- * A última linha imprime essa previsão final.

O modelo final deve ter aprendido que a relação é aproximadamente $\hat{y} \approx 1.5 \cdot x + 1.0$, então a previsão para $x=5.0$ deve estar próxima de $1.5 \cdot 5.0 + 1.0 = 8.5$.

Concluídos os testes em [app1.py](#) e [app2.py](#)

CONTINUAR Exercício 4

Explicações de IA Gemini

Executado ao [app.py](#)

Esse é um código clássico de introdução a redes neurais com Keras usando o famoso conjunto de dados **Iris**. Vou explicar o passo a passo, linha por linha e em blocos lógicos, para que você entenda exatamente o que está acontecendo.

1. Importação das Bibliotecas

Esta seção carrega as ferramentas necessárias para o projeto.

| Código | Explicação |
|---|--|
| <code>from sklearn.datasets import load_iris</code> | Importa a função para carregar o dataset Iris , um conjunto de dados padrão para classificação. |
| <code>from sklearn.model_selection import train_test_split</code> | Importa a função para dividir o dataset em conjuntos de treino e teste. |
| <code>from keras.utils import to_categorical</code> | Importa a função do Keras para realizar o one-hot encoding (codificação categórica), essencial para classificação multi-classe. |
| <code>import numpy as np</code> | Importa a biblioteca NumPy , fundamental para operações numéricas e manipulação de arrays no Python. |
| <code>from keras.models import Sequential</code> | Importa o modelo Sequential do Keras, que é a maneira mais simples de construir uma rede neural camada por camada. |
| <code>from keras.layers import Dense</code> | Importa a camada Dense (totalmente conectada), o tipo de camada mais comum em redes neurais. |

2. Preparação e Pré-processamento dos Dados

Esta é a fase onde os dados são carregados e formatados para serem usados pela rede neural.

| Código | Explicação |
|--|---|
| <code>iris = load_iris()</code> | Carrega o conjunto de dados Iris. |
| <code>X = iris.data</code> | Variáveis de Entrada (Features) : <code>X</code> recebe as características das flores (comprimento e largura da sépala, e comprimento e largura da pétala). O <code>X</code> é o que o modelo usará para aprender. |
| <code>y = iris.target</code> | Variável de Saída (Target) : <code>y</code> recebe os rótulos (classes) das espécies de Iris (0, 1 ou 2). O <code>y</code> é o que o modelo tentará prever. |
| <code>X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)</code> | Divide os dados: 80% para treino (<code>X_train</code> , <code>y_train</code>) e 20% para teste (<code>X_test</code> , <code>y_test</code>). O <code>random_state=42</code> garante que a divisão seja a mesma sempre que o código for executado. |
| <code>y_train = to_categorical(y_train)</code> | One-Hot Encoding das saídas de treino. Transforma um rótulo numérico (ex: 0, 1, 2) em um vetor binário (ex: [1, 0, 0], [0, 1, 0], [0, 0, 1]). Isso é necessário para a função de perda <code>categorical_crossentropy</code> . |
| <code>y_test = to_categorical(y_test)</code> | Aplica o mesmo One-Hot Encoding nos rótulos de teste. |

3. Definição da Arquitetura do Modelo

Aqui, a rede neural é construída camada por camada.

| Código | Explicação |
|-----------------------------------|--|
| <code>model = Sequential()</code> | Cria um modelo de rede neural sequencial , que é uma pilha linear de camadas. |

```
| `model.add(Dense(10, input_dim=4, activation='relu'))` | **Camada de Entrada/Oculto 1:** Adiciona uma camada
totalmente conectada ('Dense') com **10 neurônios**. `input_dim=4` informa que os dados de entrada têm 4 características
(as 4 medidas da flor). `activation='relu'` (Rectified Linear Unit) é a função de ativação, que introduz a não-linearidade. |
| `model.add(Dense(8, activation='relu'))` | **Camada Oculta 2:** Adiciona outra camada com **8 neurônios** e a mesma
função de ativação `relu`. |
| `model.add(Dense(3, activation='softmax'))` | **Camada de Saída:** Adiciona a camada final com **3 neurônios**, um para
cada classe (espécie de Iris). `activation='softmax'` é essencial para classificação multi-classe, pois transforma as saídas
em **probabilidades**, garantindo que a soma delas seja 1. |
```

4. Compilação e Treinamento

Nesta fase, o modelo é configurado e o processo de aprendizado é iniciado.

```
Código	Explicação
`model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])`	**Compilação:** Configura o
modelo para o treinamento:	
	- `loss='categorical_crossentropy'`: A função de **perda** (erro) usada para medir o quão bem o modelo está se saindo.
É a perda padrão para classificação multi-classe com *one-hot encoding*.	
	- `optimizer='adam'`: O algoritmo de **otimização** (Adam) que ajusta os pesos da rede para minimizar a função de
perda.	
	- `metrics=['accuracy']`: A métrica a ser monitorada (a **acurácia**).
`model.fit(X_train, y_train, epochs=150, batch_size=10, validation_split=0.1)`	**Treinamento:** Onde o aprendizado
acontece:	
	- `X_train, y_train`: Os dados de treino.
	- `epochs=150`: O número de vezes que a rede verá o conjunto de treino inteiro (150 ciclos).
	- `batch_size=10`: O número de amostras processadas antes que os pesos do modelo sejam atualizados (lote de 10).
	- `validation_split=0.1`: Reserva **10%** dos dados de treino para **validação** durante o treinamento, para monitorar o
desempenho. |
```

5. Avaliação e Previsão

Por fim, o modelo treinado é testado em dados novos e são feitas previsões.

```
Código	Explicação
`_ , accuracy = model.evaluate(X_test, y_test)`	**Avaliação:** O modelo é testado nos dados de teste ('X_test', 'y_test')
que ele *nunca viu* para medir sua performance no mundo real. O resultado da avaliação inclui a perda e a acurácia.	
`print(f'Acurácia: {accuracy*100:.2f}%)`	Exibe a acurácia calculada no conjunto de teste.
`predictions = np.argmax(model.predict(X_test), axis=1)`	**Previsão:** Pede ao modelo para prever as classes das flores
no conjunto de teste.	
	- `model.predict(X_test)` retorna as **probabilidades** para cada uma das 3 classes (ex: [0.01, 0.98, 0.01]).
	- `np.argmax(..., axis=1)` encontra o **índice** da probabilidade mais alta em cada linha. Este índice (0, 1 ou 2) é a
classe prevista.	
`y_test_labels = np.argmax(y_test, axis=1)`	Converte os rótulos de teste **one-hot encoded** ([1, 0, 0]) de volta para a
sua representação numérica original (0), para facilitar a comparação com as previsões.	
`species = {'0': 'Iris setosa', 1: 'Iris versicolor', 2: 'Iris virginica'}`	Cria um dicionário para mapear os índices numéricos das
classes para seus nomes reais.	
`predicted_class = predictions[0]`	Pega a classe prevista para a primeira amostra do conjunto de teste.
`true_class = y_test_labels[0]`	Pega a classe real da primeira amostra do conjunto de teste.
`print(...)`	Exibe a previsão e o valor real da primeira amostra como um exemplo prático.
```


Executado ao [main.py](#)

Explicações da Gemini IA

O código implementa uma **Rede Neural Convolutacional (CNN) 1D** para a tarefa de **classificação de notícias em tópicos** usando o famoso dataset **Reuters**, que possui 46 categorias diferentes.

1\ Importações e Configurações Iniciais

```
```python
import tensorflow as tf
from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Embedding, GlobalMaxPooling1D
from tensorflow.keras.preprocessing.sequence import pad_sequences
```
```

Esta seção importa todas as bibliotecas e módulos necessários do **TensorFlow** e **Keras**:

- tensorflow as tf**: A biblioteca principal para aprendizado de máquina.
- reuters**: O dataset de notícias que será usado para treinamento.
- Tokenizer**: Uma classe para vetorizar dados de texto. Embora esteja importada, ela **não é explicitamente usada** neste código para a tokenização (o dataset `reuters.load_data` já retorna sequências de inteiros), mas é uma importação comum em projetos de NLP.
- Sequential**: Define um modelo onde as camadas são empilhadas sequencialmente.
- Dense**: Uma camada densamente conectada (fully connected).
- Dropout**: Uma técnica de regularização para evitar overfitting, desativando aleatoriamente neurônios durante o treinamento.
- Embedding**: Uma camada essencial para NLP que mapeia números inteiros (índices de palavras) em vetores densos de ponto flutuante.
- GlobalMaxPooling1D**: Reduz a dimensionalidade da saída da camada de **embedding**.
- pad_sequences**: Garante que todas as sequências (notícias) de entrada tenham o mesmo comprimento, preenchendo ou truncando conforme necessário.

2\ Carregamento e Pré-processamento dos Dados

```
```python
Carregar os dados do dataset Reuters
max_words = 10000 # Considerar apenas as 10.000 palavras mais frequentes
max_len = 200 # Considerar apenas as primeiras 200 palavras de cada texto
```
```

```
(x_train, y_train), (x_test, y_test) = reuters.load_data(num_words=max_words)
```

1. `max_words = 10000`: Define que apenas as **10.000** palavras mais frequentes no dataset serão consideradas. Palavras raras serão descartadas para reduzir a complexidade e o ruído.
2. `max_len = 200`: Define que cada notícia (sequência de palavras) será limitada a um comprimento máximo de **200** palavras/tokens. Notícias mais longas serão truncadas e as mais curtas serão preenchidas (no próximo passo).
3. `reuters.load_data(num_words=max_words)`: Carrega o dataset.
 - * `x_train` e `x_test`: As notícias, já representadas como sequências de **inteiros** (onde cada inteiro é o índice de uma palavra no vocabulário de `max_words`).
 - * `y_train` e `y_test`: Os rótulos (categorias), que são números inteiros de **0 a 45** (46 categorias).

<!-- end list -->

```
python
```

```
# Tokenizar os dados (embora o load_data já entregue os índices, essa linha não faz nada)
tokenizer = Tokenizer(num_words=max_words)
x_train = pad_sequences(x_train, maxlen=max_len)
x_test = pad_sequences(x_test, maxlen=max_len)
```

1. `Tokenizer(...)`: A linha que cria o objeto `Tokenizer` é redundante, pois a próxima etapa lida com a uniformização do comprimento das sequências já existentes.
2. `pad_sequences(...)`: Essa é a etapa crucial. Ela transforma as sequências de tamanhos variados em matrizes de tamanho uniforme (`max_len=200`).
 - * Se uma notícia tem menos de 200 palavras, zeros são adicionados ao início (`padding='pre'`, que é o padrão).
 - * Se tem mais de 200, é truncada.
 - * O resultado é que `x_train` e `x_test` agora são matrizes onde cada linha tem exatamente 200 colunas.

3\ Construção do Modelo (CNN 1D para Texto)

```
python
```

```
# Construir o modelo
model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=128, input_length=max_len))
model.add(Dropout(0.2))
model.add(GlobalMaxPooling1D())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(46, activation='softmax'))
```

Este é o coração da rede neural:

1. `model = Sequential()`: Inicia o modelo sequencial.

2. `Embedding(...)`: A primeira camada.
`input_dim=max_words` (10000)`: O tamanho do vocabulário.
`output_dim=128``: Define que cada palavra será mapeada para um vetor de 128 dimensões.
`input_length=max_len` (200)`: O comprimento da sequência de entrada (cada notícia).
`Saída desta camada`: Uma matriz de formato `(None, 200, 128)``.
3. `Dropout(0.2)`: Desativa 20% dos neurônios da camada de `embedding` aleatoriamente.
4. `GlobalMaxPooling1D()`: Pega o valor máximo (o "feature" mais relevante) ao longo de cada vetor de `embedding` (a dimensão 1D). Isso condensa a matriz de saída para um único vetor longo para cada notícia.
`Saída desta camada`: Um vetor de formato `(None, 128)``.
5. `Dense(128, activation='relu')`: Uma camada densamente conectada com 128 neurônios e a função de ativação `ReLU` (Rectified Linear Unit).
6. `Dropout(0.2)`: Desativa 20% dos neurônios desta camada densa.
7. `Dense(46, activation='softmax')`: A camada de saída.
`46`: O número de neurônios é 46, correspondendo ao número de classes/categorias do dataset Reuters.
`softmax`: A função de ativação que converte os `scores` em `probabilidades` que somam 1. A categoria com a maior probabilidade é a previsão do modelo.

4\. Compilação, Treinamento e Avaliação

```
python
# Compilar o modelo
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# Treinar o modelo
history = model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

# Avaliar o modelo
score = model.evaluate(x_test, y_test, batch_size=32)
print(f'Test accuracy: {score[1]}')
```

1. `model.compile(...)`: Configura o processo de aprendizado:
`optimizer='adam'`: O algoritmo de otimização que ajusta os pesos da rede.
`loss='sparse_categorical_crossentropy'`: A função de perda. É usada porque os rótulos (`y_train`) são inteiros (0, 1, 2, ...) e não foram convertidos para `one-hot encoding`.
`metrics=['accuracy']`: A métrica que será monitorada (a porcentagem de acertos).
2. `model.fit(...)`: Treina o modelo:
`x_train, y_train`: Dados de treinamento.
`epochs=10`: O modelo passará por todo o conjunto de treinamento 10 vezes.
`batch_size=32`: 32 amostras serão processadas de cada vez antes de atualizar os pesos.
`validation_split=0.2`: 20% dos dados de `x_train` serão separados automaticamente para validação durante o treinamento (para monitorar o `overfitting`).

3. `model.evaluate(...)`: Avalia o desempenho final do modelo no conjunto de teste (`x_test`, `y_test`), que são dados nunca vistos.
* `score[1]` é a precisão (accuracy) no conjunto de teste.

5\. Fazendo Previsões

```
python
# Fazer previsões
predictions = model.predict(x_test)

# Exemplo de previsão para a primeira notícia do conjunto de teste
predicted_label = predictions[0].argmax()
print(f'Predicted label: {predicted_label}')
print(f'True label: {y_test[0]}')
```

1. `predictions = model.predict(x_test)`: O modelo gera as previsões para todas as notícias no conjunto de teste.
* A saída é uma matriz onde cada linha é um vetor de 46 probabilidades (a saída da camada `softmax`).
2. `predictions[0].argmax()`: Pega o primeiro vetor de probabilidades (`predictions[0]`) e encontra o `índice` com a `maior probabilidade` (`argmax()`). Esse índice é o rótulo da classe prevista.
3. As linhas finais imprimem o rótulo previsto e o rótulo real (correto) para a primeira notícia de teste, permitindo uma inspeção rápida do resultado.

Continuar no exemplo 05

- Além dos requirement.txt foi necessário instalar o Ollama.
- A instalação do Ollama não é feita via pip
- Eu consegui rodar o código do professor que deu alguns warnings;
- Eu aproveitei e fiz um outro código usando um exemplo do da Gemini IA

Instalações e pré-requisitos

Instalação do Ollama

```
sudo curl -fsSL https://ollama.com/install.sh | sh
```

Testar a instalação

```
ollama --version
```

```
ollama version is 0.12.3
```

```
sudo systemctl status ollama
```

```
ollama
● ollama.service - Ollama Service
   Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-01 21:08:34 -03; 5min ago
     Main PID: 441435 (ollama)
       Tasks: 8 (limit: 18964)
      Memory: 35.4M (peak: 36.3M)
         CPU: 87ms
    CGroup: /system.slice/ollama.service
            └─441435 /usr/local/bin/ollama serve

out 01 21:08:35 brunojose-Al3 ollama[441435]: time=2025-10-01T21:08:35.101-03:00 level=INFO source=gpu.go:217 msg="lookin
out 01 21:08:35 brunojose-Al3 ollama[441435]: time=2025-10-01T21:08:35.119-03:00 level=WARN source=amd_linux.go:61 msg="c
out 01 21:08:35 brunojose-Al3 ollama[441435]: time=2025-10-01T21:08:35.120-03:00 level=INFO source=amd_linux.go:406 msg="
out 01 21:08:35 brunojose-Al3 ollama[441435]: time=2025-10-01T21:08:35.120-03:00 level=INFO source=gpu.go:396 msg="no com
out 01 21:08:35 brunojose-Al3 ollama[441435]: time=2025-10-01T21:08:35.120-03:00 level=INFO source=types.go:131 msg="infe
out 01 21:08:35 brunojose-Al3 ollama[441435]: time=2025-10-01T21:08:35.120-03:00 level=INFO source=routes.go:1569 msg="er
out 01 21:09:44 brunojose-Al3 ollama[441435]: [GIN] 2025/10/01 - 21:09:44 | 200 | 919.793188ms | 127.0.0.1 | GET
out 01 21:09:44 brunojose-Al3 ollama[441435]: [GIN] 2025/10/01 - 21:09:44 | 404 | 6.81µs | 127.0.0.1 | GET
out 01 21:11:45 brunojose-Al3 ollama[441435]: [GIN] 2025/10/01 - 21:11:45 | 200 | 29.565µs | 127.0.0.1 | GET
out 01 21:13:43 brunojose-Al3 ollama[441435]: [GIN] 2025/10/01 - 21:13:43 | 200 | 256.315µs | 127.0.0.1 | GET
lines 1-20/20 (END)
```

Acesso web

The Ollama API is now available at

<http://127.0.0.1:11434>

```
ollama is running
```

Instalando o client via pip

```
pip install -U langchain-community  
pip install langchain langchain-community langchain-chroma
```

Instalação do Modelo LLama

```
ollama run llama2
```

obs

Ficou bem lento no meu laptop

Um pouco de conhecimento sobre RAG

RAG significa "**Retrieval-Augmented Generation**" (Geração Aumentada por Recuperação), e é uma técnica muito popular no campo da Inteligência Artificial, especialmente com os **Grandes Modelos de Linguagem (LLMs)**, como o que está a ser usado para gerar esta resposta.

Em termos simples, o RAG é um *framework* que permite aos LLMs aceder e usar informações **externas** àquelas com que foram originalmente treinados, para gerar respostas mais precisas, atualizadas e com base em factos específicos.

Como o RAG funciona?

Imagine que um LLM é um estudante muito inteligente, mas que só consegue responder com base no que leu até ao seu último exame (os dados de treino). O RAG é como dar a esse estudante a capacidade de **consultar livros e documentos** relevantes (uma base de conhecimento externa) *antes* de responder a uma pergunta.

O processo geralmente segue estes passos:

1. **Recuperação (Retrieval):** Quando um utilizador faz uma pergunta, o sistema RAG primeiro pesquisa numa base de conhecimento externa (que pode ser documentos da sua empresa, um banco de dados, ou páginas web) para encontrar os pedaços de informação mais **relevantes** para essa pergunta. Essa informação é geralmente

convertida em um formato numérico chamado *embeddings* e armazenada num *vector database* para pesquisa rápida.

2. **Aumento (Augmentation):** A informação relevante encontrada na base de conhecimento é então **adicionada** à pergunta original do utilizador. Isso cria um *prompt* "aumentado" ou enriquecido.
3. **Geração (Generation):** O *prompt* aumentado (a pergunta do utilizador + o contexto relevante) é enviado para o LLM. Agora, o modelo usa esse novo contexto para **gerar a resposta**, em vez de depender apenas da sua memória interna.

Por que o RAG é importante?

O RAG resolve algumas limitações chave dos LLMs tradicionais:

- **Evita a "Alucinação":** LLMs podem ocasionalmente inventar factos (conhecido como "alucinação"). O RAG **ancora** a resposta em fontes de dados reais e verificáveis, o que aumenta a precisão e a fiabilidade.
- **Acesso a Informação Atualizada:** Os LLMs têm conhecimento limitado até a data do seu último treino. O RAG permite que o modelo aceda a **dados em tempo real** ou muito recentes, sem a necessidade de dispendiosos e demorados retreinos do modelo base.
- **Conhecimento Específico de Domínio:** Empresas podem usar o RAG para conectar um LLM generalista aos seus **documentos internos**, manuais de produtos, FAQs, ou bases de dados, tornando o modelo útil para tarefas muito específicas da organização.
- **Redução de Custos:** É muito mais barato e rápido implementar um sistema RAG do que treinar um LLM do zero ou fazer *fine-tuning* (ajuste fino) frequente para incluir novos dados.

Em resumo, o RAG é uma forma poderosa e eficiente de tornar os LLMs **mais úteis, precisos, e relevantes** para aplicações do mundo real.